Universidade Federal do Pará

Faculdade de Engenharia da Computação e Telecomunicações

Breno de Medeiros Paulo - 202206840034
Danilo Augusto de Assis Castelo Branco - 201706840027
Leonardo da Silva Pinheiro - 202206840040
Nicolas Ranniery da Silva Sousa - 202206840051

Fevereiro 2026

Projeto Final

1 Introdução e Objetivo

O objetivo principal deste projeto é investigar e quantificar como a variação de hiperparâmetros afeta a estabilidade, a velocidade de convergência e o comportamento final de um agente de Aprendizado por Reforço.O ambiente escolhido foi o MountainCarContinuous-v0, onde o agente deve conduzir um carro ao topo de uma montanha ajustando a força do motor. O algoritmo utilizado para realizar as comparações de hiperparametros é o PPO (Proximal Policy Optimization) da biblioteca Stable-Baselines3.

2 Ambiente de execução

Para realizar os treinamentos, foi necessário preparar o ambiente de execução, a versão do Python utilizada foi a **3.13.12** e todos os comandos foram executados no **Command Prompt do Windows**. Além disso, diversas bibliotecas foram instaladas e que possuem funções diferentes no treinamento

Núcleo do Aprendizado

- stable-baselines3: A biblioteca principal que contém as implementações dos algoritmos (PPO)
- torch (PyTorch): Responsável pelos cálculos da Rede Neural
- gymnasium: A interface padrão que define como o agente interage com o ambiente
- numpy: Manipulação de matrizes e vetores matemáticos

Simulação

- pygame: Responsável pela renderização gráfica

Processamento de Imagem

- optuna: Biblioteca para otimização de hiperparâmetros.
- sb3-contrib: Extensões e algoritmos experimentais da Stable Baselines.
- huggingface sb3: Utilizados para salvar e carregar modelos.

3 Treinamento

Para este estudo, foram selecionados dois hiperparametros que serão modificados:

Taxa de Aprendizado (Learning Rate): Define o tamanho do passo que o otimizador dá ao atualizar os pesos da rede neural. Valores muito altos podem causar instabilidade; muito baixos tornam o aprendizado lento

Fator de Desconto (Gamma): Determina a importância das recompensas futuras. Varia de 0 a 1. Próximo a 1 torna o agente focado em recompensas de longo prazo; próximo a 0 torna-o limitado.

As comparações e modificações de hiperparametros realizadas foram as seguintes:

- Treinamento PPO padrão
- Treinamento PPO learning-rate baixo
- Treinamento PPO learning-rate alto
- Treinamento PPO gamma baixo
- Treinamento PPO learning-rate alto e gamma baixo

É importante ressaltar que todos os treinamentos foram configurados para **50.000 passos** e que a cada **2.500 passos** de treino, o agente pare de aprender e jogue **10 partidas** do jogo apenas com o que já sabe, sem atualizar os pesos matemáticos.

3.1 Treinamento PPO padrão

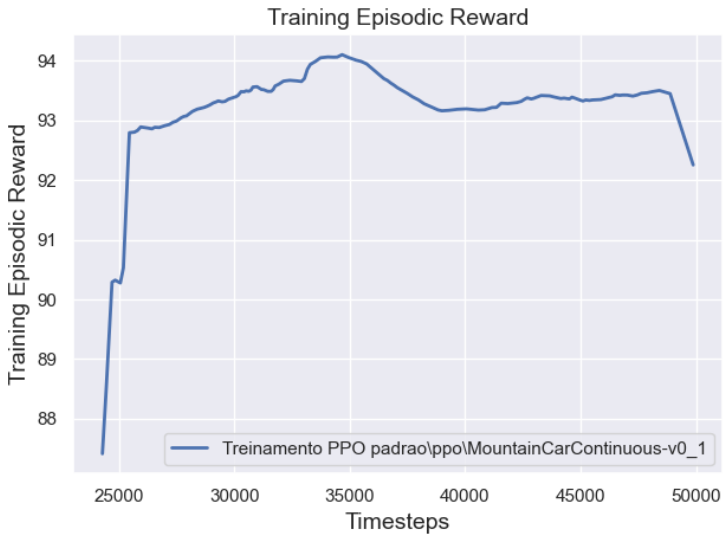
O treinamento PPO padrão foi realizado e gravado na pasta **Treinamento PPO padrão**. A seguinte linha de código foi executada para iniciar o treino:

```
python train.py --algo ppo --env MountainCarContinuous-v0 -f "
 Treinamento PPO padrao" --eval-freq 2500 --eval-episodes 10 -n 50000
```

Os hiperparametros considerados padrões são:

- Learning Rate: 0.00007
- Gamma: 0.9999

Após a execução, os dados de treino foram salvos na própria pasta



O seguinte comando foi executado para plotar o gráfico:

```
python rl_zoo3/plots/plot_train.py -a ppo -e MountainCarContinuous-v0
 -f "Treinamento PPO padrao" -x steps
```

O gráfico exibe uma ascensão quase vertical próximo ao passo 25.000. Isso reflete o momento em que o agente, compreende a física do sistema. Ele descobre que precisa acelerar no sentido oposto ao objetivo para superar a gravidade da montanha, resultando no salto abrupto da recompensa de aproximadamente 87 para 93.

Na documentação oficial do Gymnasium, este ambiente é considerado "resolvido" quando o agente atinge uma pontuação média superior a 90. O gráfico demonstra que o modelo atinge essa meta rapidamente e estabiliza, flutuando entre 93 e 94. Essa curva suave e constante comprova que a taxa de aprendizado (learning rate) e o fator de desconto (gamma) padrão estão perfeitamente ajustados.

Observa-se uma leve queda na recompensa nos instantes finais. Esse comportamento ocorre pois o agente realiza pequenas variações no controle do carro para testar se existe uma rota ainda mais eficiente, o que ocasionalmente resulta em perdas de pontuação.

3.2 Treinamento PPO learning rate baixo

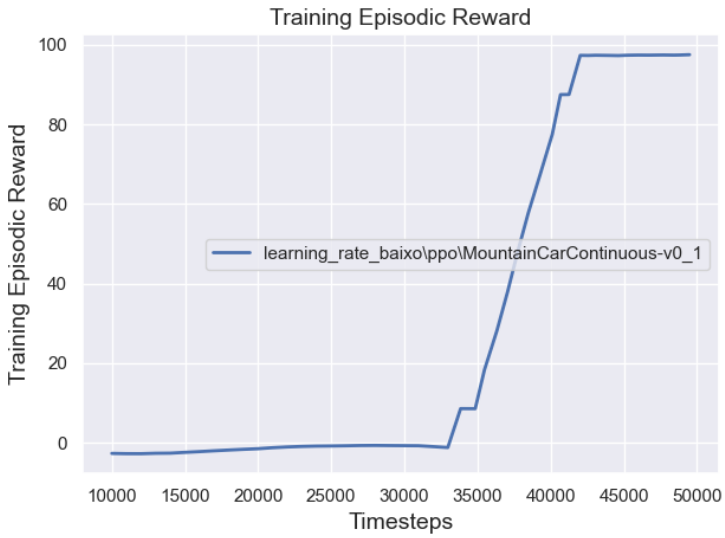
O Treinamento PPO learning-rate baixo foi realizado e gravado na pasta **learning rate baixo**. A seguinte linha de código foi executada para iniciar o treino:

```
python train.py --algo ppo --env MountainCarContinuous-v0 -f "
 learning_rate_baixo" --eval-freq 2500 --eval-episodes 10 -n 50000 --
 hyperparams learning_rate:0.000001
```

Os hiperparametros utilizados foram:

- Learning Rate: 0.000001
- Gamma: 0.9999

Após a execução, os dados de treino foram salvos na própria pasta



O seguinte comando foi executado para plotar o gráfico:

```
python rl_zoo3/plots/plot_train.py -a ppo -e MountainCarContinuous-v0
 -f "learning_rate_baixo" -x steps -w 10
```

Enquanto o modelo Padrão iniciou sua subida de aprendizado por volta dos 25.000 passos, este modelo passou os primeiros 33.000 passos com uma recompensa próxima a zero ou negativa. A rede neural demorou muito mais tempo para acumular as alterações de pesos necessárias para iniciar o movimento de pêndulo do carro.

A partir dos 33.000 passos, quando o agente finalmente compreende o objetivo, a curva de aprendizado sobe de forma quase reta e sem nenhum ruído. Ao atingir a marca dos 42.000 passos, a pontuação estabiliza no topo (acima de 90) formando uma linha perfeitamente reta até o final do treinamento.

Isso comprova que uma taxa de aprendizado baixa elimina completamente o risco de "esquecimento"por parte da rede neural, tornando a recompensa final extremamente confiável, embora o processo até chegar lá seja ineficiente e lento.

3.3 Treinamento PPO learning-rate alto

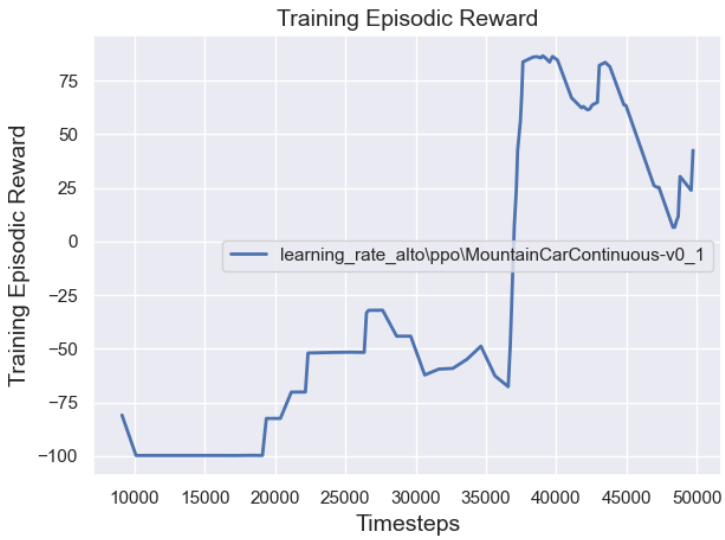
O Treinamento PPO l alto foi realizado e gravado na pasta **learning rate alto**. A seguinte linha de código foi executada para iniciar o treino:

```
python train.py --algo ppo --env MountainCarContinuous-v0 -f "
 learning_rate_alto" --eval-freq 2500 --eval-episodes 10 -n 50000 --
 hyperparams learning_rate:0.01
```

Os hiperparametros utilizados foram:

- Learning Rate: 0.01
- Gamma: 0.9999

Após a execução, os dados de treino foram salvos na própria pasta



O seguinte comando foi executado para plotar o gráfico:

```
python rl_zoo3/plots/plot_train.py -a ppo -e MountainCarContinuous-v0
 -f "learning_rate_alto" -x steps -w 10
```

Este gráfico demonstra uma alta instabilidade na manutenção da boa recompensa. Observa-se que o agente passa os primeiros 20.000 passos com desempenho mínimo (-100). Ao contrário do modelo padrão que sobe de uma vez, este modelo apresenta "degraus"de aprendizado instáveis, refletindo atualizações muito agressivas que dificultam a estabilização em pontuações intermediárias.

Entre os passos 35.000 e 40.000, o modelo finalmente descobre a mecânica do pêndulo e a pontuação dispara para níveis aceitáveis (próximos a 80). No entanto, o traço mais importante do gráfico ocorre logo em seguida: uma grande queda da recompensa média nos 10.000 passos finais.

Essa queda comprova que a atualização dos pesos da rede neural é grande demais. Em vez de estacionar no mínimo local ideal , o gradiente "passa do ponto"(overshooting), fazendo com que o agente desaprenda o que havia acabado de descobrir, terminando o treinamento com uma recompensa mediana.

3.4 Treinamento gamma baixo

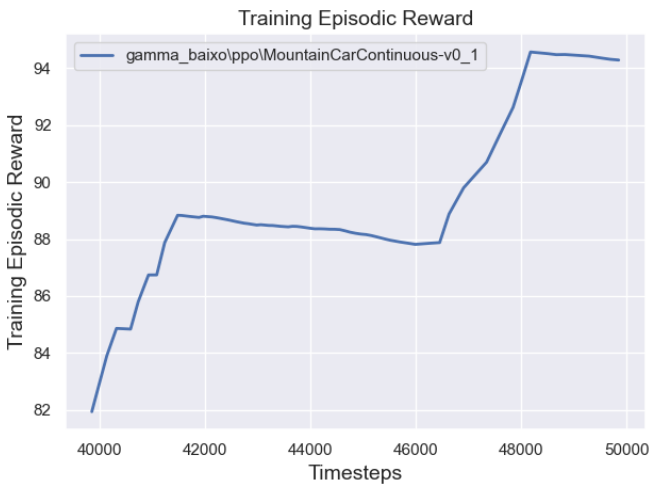
O Treinamento PPO gamma baixo foi realizado e gravado na pasta **gamma baixo**. A seguinte linha de código foi executada para iniciar o treino:

```
python train.py --algo ppo --env MountainCarContinuous-v0 -f "
 gamma_baixo" --eval-freq 2500 --eval-episodes 10 -n 50000 --
 hyperparams gamma:0.80
```

Os hiperparametros utilizados foram:

- Learning Rate: 0.00007
- Gamma: 0.80

Após a execução, os dados de treino foram salvos na própria pasta



O seguinte comando foi executado para plotar o gráfico:

```
python rl_zoo3/plots/plot_train.py -a ppo -e MountainCarContinuous-v0
 -f "gamma_baixo" -x steps
```

O gráfico acima ilustra o impacto direto da redução do fator de desconto na capacidade de planejamento a longo prazo do agente. O modelo com Gamma baixo apresentou um atraso em seu aprendizado, com a curva de recompensa surgindo apenas após a marca de 40.000 passos.

O ambiente MountainCar exige que o agente se afaste do objetivo para conseguir o impulso necessário para a vitória. Com um Gamma de 0.80, o algoritmo desvaloriza drasticamente as recompensas futuras, tornando o agente limitado.

Ele resiste à estratégia de recuar, pois isso não oferece um benefício matemático imediato. O agente passou a maior parte do treinamento preso em mínimos locais no fundo do vale, esgotando o limite de tempo dos episódios iniciais. Apenas nos 10.000 passos finais o modelo conseguiu atingir o objetivo, escalando rapidamente a pontuação para a faixa de 94 pontos.

3.5 Treinamento learning rate alto and gamma baixo

O Treinamento PPO learning rate alto and gamma baixo foi realizado e gravado na pasta **learning rate alto and gamma baixo**. A seguinte linha de código foi executada para iniciar o treino:

```
python train.py --algo ppo --env MountainCarContinuous-v0 -f "
 learning_rate_alto and gamma_baixo" --eval-freq 2500 --eval-episodes
 10 -n 50000 --hyperparams learning_rate:0.01 gamma:0.80
```

Os hiperparametros utilizados foram:

- Learning Rate: 0.01
- Gamma: 0.80

Após a execução, os dados de treino foram salvos na própria pasta

O seguinte comando foi executado para plotar o gráfico:

```
python rl_zoo3/plots/plot_train.py -a ppo -e MountainCarContinuous-v0
 -f "learning_rate_alto and gamma_baixo" -x steps -w 10
```

O gráfico acima evidencia o colapso do aprendizado quando submetida a hiperparâmetros opostos ao ideal. A curva apresenta um comportamento altamente oscilatório, caracterizado por picos de alta recompensa seguidos por quedas abruptas (atingindo valores negativos de até -75).

Os vales profundos logo após os picos de sucesso comprovam o fenômeno de overshooting. Devido à taxa de aprendizado alta ($\alpha = 0.01$), as atualizações nos pesos da rede neural são tão agressivas que o modelo destrói imediatamente os aprendizados bem-sucedidas recém-descobertos.

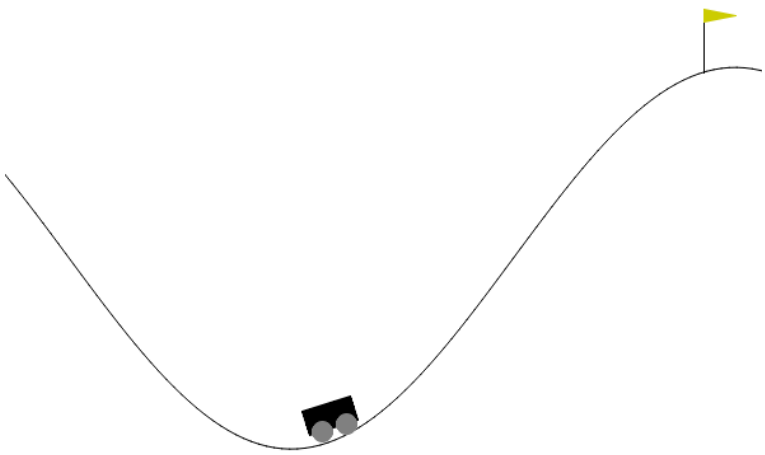
O fator de desconto baixo ($\gamma = 0.80$) impede que o agente compreenda a sequência de ações que o levou ao sucesso. Ele não consegue correlacionar o impulso inicial para trás com a chegada à bandeira. A partir dos 40.000 passos, a rede neural perde completamente a capacidade de convergência.

4 Videos

Foram gravados trechos de jogabilidade de aproximadamente 30 segundos (1000 frames). Os arquivos resultantes foram salvos automaticamente na pasta videos/ em formato MP4:

```
python enjoy.py --algo ppo --env  MountainCarContinuous-v0 -f "NOME DA
  PASTA"
```

Listing 1: Comando para reproduzir os videos



```
python -m rl_zoo3.record_video --algo ppo --env  MountainCarContinuous
 -v0 -f "NOME DA PASTA" -n 1000
```

Listing 2: Comando para salvar os videos